← Course home L4 slides →

Lecture 4 — GEMM: from formulation to a customized FPGA kernel

Student handout · FPGA-based Accelerator Design · Monsoon 2026

Scope. GEMM — general matrix multiply, $C = AB$ — finally enters the course as a first-class object. Lectures 1–3 taught the datapath, the pipeline, and the loop-carried recurrence on a **single dot product**; this lecture scales that dot product up to a real kernel and then *makes it fast*. We do it as a **design-space walk**: one triple-loop nest, **four strategies** — naive → pipe-k → pipe-j → row-wise — each read for the **one wall it hits**, each wall naming the next move. Two walls appear at GEMM scale: a familiar **data** hazard (the float recurrence from L3) and a new **structural / memory-port** hazard. A change of *dataflow* tears both down to **II = 1 on float** (measured), and then **width** (array partitioning / reshaping) spends the reuse it bought. Every number is synthesized for the U50 (Vitis 2025.1, 32^3), cosim-verified. This handout follows the Lecture 4 slides.

> **Thesis (the spine).** Matrix multiply is one triple loop but **not one algorithm.** The *order* you sweep it and the *shape* you slice it into decide what you **reuse** versus **re-fetch**; on hardware, *which loop you pragma decides which hazard you hit*. Same $2N^3$ flops throughout — the memory bill and the II are a **dataflow choice**, made before the pragma and merely exposed by it. Performance is earned by changing the dataflow, not by adding a directive.

Learning outcomes

By the end of this lecture you can:

1. State GEMM's cost (MNK MACs $\approx 2N^3$ flops over $3N^2$ words) and derive its **arithmetic-intensity ceiling $O(N)$**, versus the **$O(1)$** a naive kernel achieves.
2. Write the **three formulations** (inner-product / outer-product / row-wise) and the **six loop orders**, and apply the one rule that ranks them: *the operand whose index set omits the innermost loop variable is the one reused / broadcast.*
3. Explain how off-chip memory **delivers** operands (transactions, bursts, port width) and why **layout precedes parallelism**.
4. Read a synthesis report and the HLS log to explain the **naive floor (II = 32)** as a **structural / memory-port** hazard on B — *not* the reduction — and explain **loop coalescing** (`rows_cols`).
5. Predict and read off why "just add `PIPELINE` " gives **two different failures** depending on *which loop* you pragma: a **data** hazard (pipe-k) and a **structural** hazard (pipe-j); and write achieved II as a **max of three bounds**.
6. Explain the **row-wise** dataflow that reaches **II = 1 on float** — capturing reuse *and* dissolving the recurrence — and why the two fixes turn out to be the same move.
7. Use `ARRAY_PARTITION` and `ARRAY_RESHAPE` (cyclic / block / complete) correctly, read the **measured width sweep**, and explain why the recurrence *returns* past the knee; and know when **tiling** becomes necessary.

Running example — the dot product, wrapped into GEMM

L1 built it, L2 pipelined it, L3 made it survive a float recurrence. It is the **GEMM inner product**; L4 wraps it in the two loops that walk every output:

```
// C = A·B     A:M×K   B:K×N   C:M×N   (row-major)
rows: for (int i = 0; i < M; i++)
  cols: for (int j = 0; j < N; j++) {
      acc_t acc = 0;
      prod: for (int k = 0; k < K; k++)
          acc += A[i*K+k] * B[k*N+j];      // ← L1/L2/L3's dot product, verbatim
      C[i*N+j] = acc;
  }
```

Throughout we take the concrete sizes $M = N = K = 32$ and `data_t = float`.

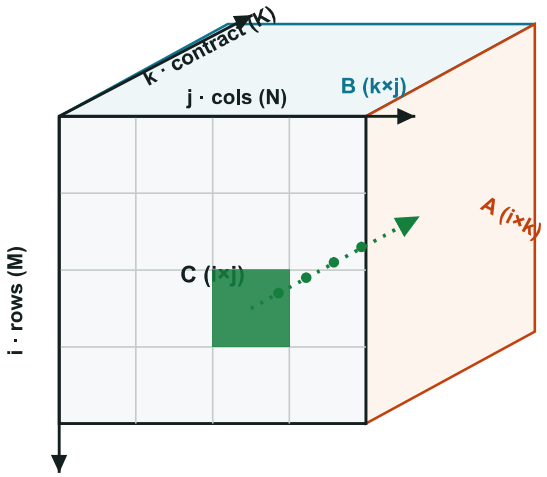
Part I — What GEMM is, and the reuse it makes available

For a square problem of size N, GEMM does $\approx 2N^3$ **flops** but touches only $3N^2$ **words**. The **ideal arithmetic intensity** (L2's roofline x-axis) is

$$\text{AI}_\text{ceiling} = \frac{2N^3}{3N^2} = \tfrac{2}{3}N = O(N).$$

Each word *can* take part in $O(N)$ operations — enough reuse to be deeply **compute-bound**, *if* every word is fetched once and reused on-chip. A **naive** kernel re-reads its operands $O(N)$ times, dragging *achieved* AI down to $O(1)$ — **memory-bound**. Same flops; the whole lecture is about closing the gap between $O(N)$ and $O(1)$, and it is a dataflow-and-buffering choice.

GEMM is a 3-D iteration space — one loop nest over (i, j, k)



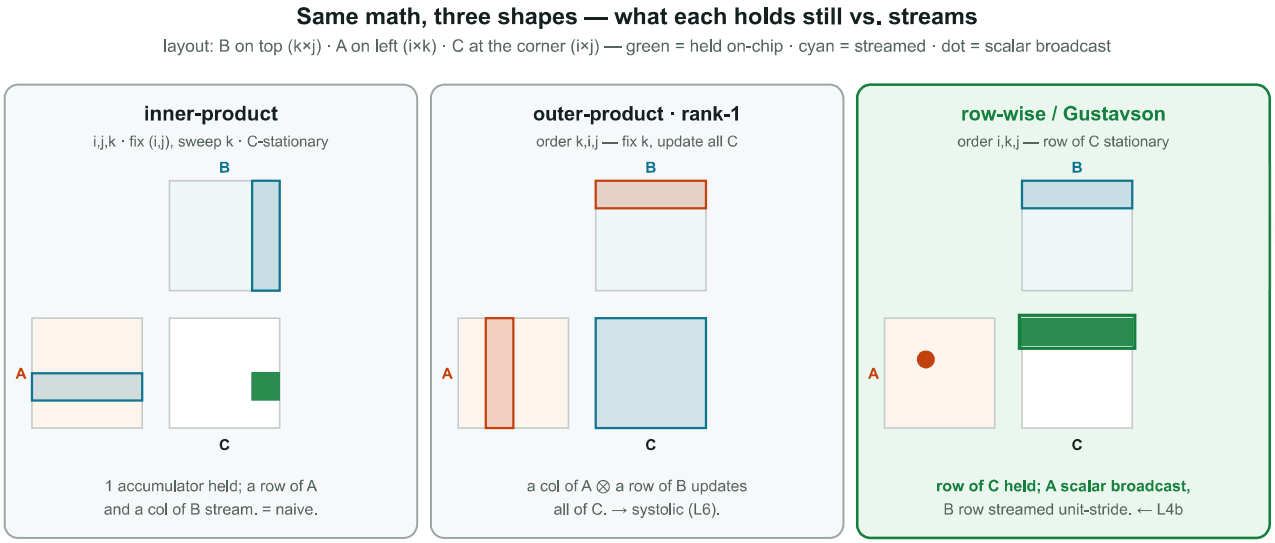
one output C[i][j]
= a **ray along k** — the dot product of row i of A and col j of B.

The whole cube = M·N·K multiply–adds. How you **sweep** it (Part II–III) sets what you reuse — **B on top**, **A on the side**.

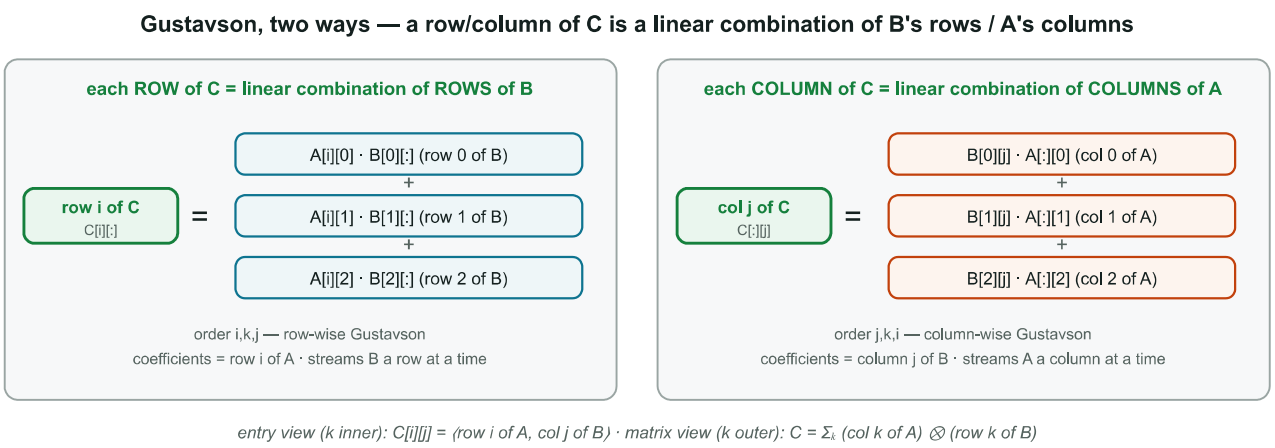
Part II — Same math, three shapes

The triple sum can be evaluated in any order; three orderings are canonical because each holds a different object **stationary** — and "stationary" is exactly "kept on-chip and reused."

- **Inner-product** (i,j,k) — fix (i,j), sweep k. One accumulator is stationary; streams a row of A and a **column** of B (stride-N). **This is the naive GEMM.**
- **Outer-product** (k,i,j) — fix k, sweep (i,j). Each k does a **rank-1 update of all of C**. This is the systolic form (a later lecture).
- **Row-wise / Gustavson** (i,k,j) — fix i. A **row of C** is stationary; a broadcast scalar $A[i][k]$ scales a streamed row of B (unit stride). **The FPGA winner**, and the order CPU BLAS uses innermost.



Read at three granularities, these are the same identity: an **entry** of C is a dot product; a **row** (or column) of C is a linear combination of the rows of B (or columns of A) — the Gustavson view; and the **whole matrix** is a sum of K rank-1 outer products.

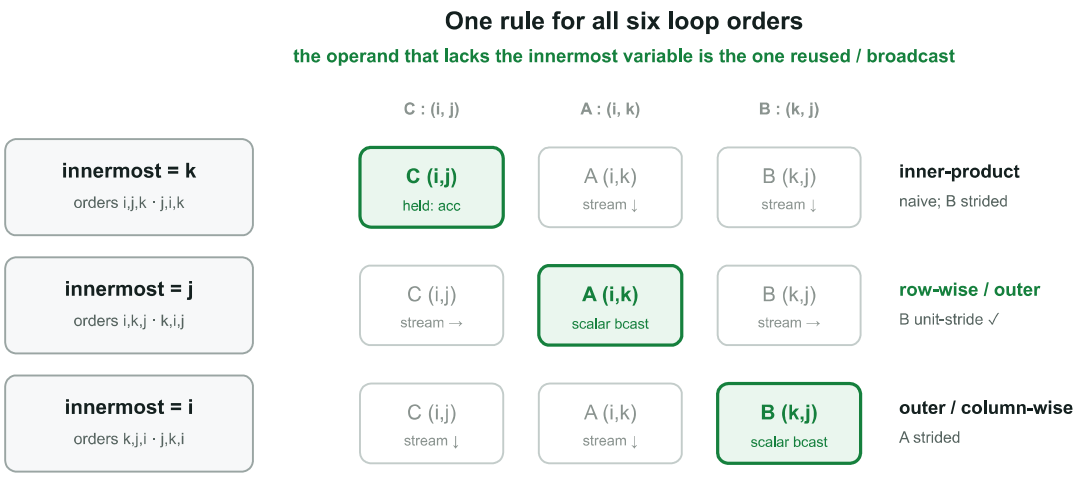


entry view (k inner): $C[i][j] = \langle$(row i of A, col j of B)$\rangle$ · matrix view (k outer): $C = \Sigma_k$ (col k of A) $\otimes$ (row k of B)

Part III — Six loop orders, one taxonomy

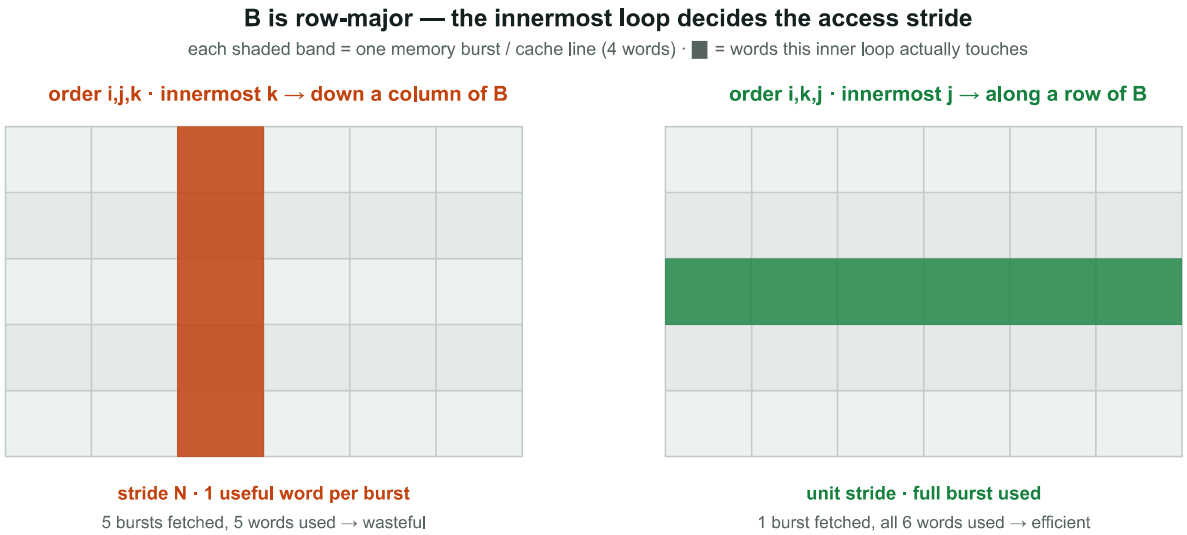
The three shapes are three *views*; the six loop orders are the concrete schedules, and they fall out of **two independent choices**. **(1) Where k sits** picks the product view: k innermost → dot; k middle → Gustavson; k outermost → outer. **(2) The order of i and j** picks row- versus column-major traversal.

The one rule that ranks them for reuse: **the operand whose index set omits the innermost loop variable is the one held / broadcast.** Innermost k → the C accumulator is reused; innermost j → the $A[i][k]$ scalar is broadcast; innermost i → the $B[k][j]$ scalar is broadcast.



Three index sets, three cases — the whole reuse theory of GEMM.

Locality follows from the same choice, straight from CPU-cache intuition: **innermost `k` walks B down a column** (stride N — poor); **innermost `j` walks it along a row** (unit stride — good). Stride decides locality *before* any reuse argument, and it is why the middle-k (Gustavson) orders suit streaming hardware.

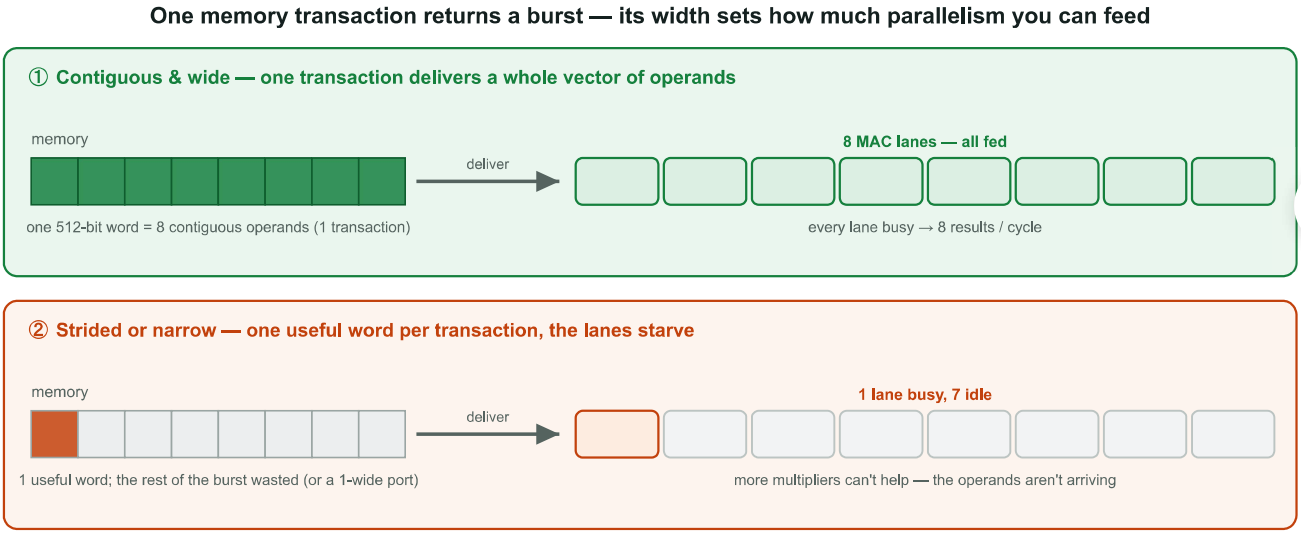


Same lesson on any tiered memory — cache line, HBM burst, DRAM page. On the FPGA (L4b) the "cache" is on-chip BRAM you manage.

Part IV — GEMM on the FPGA: how memory feeds compute, and the naive floor

How memory delivers operands

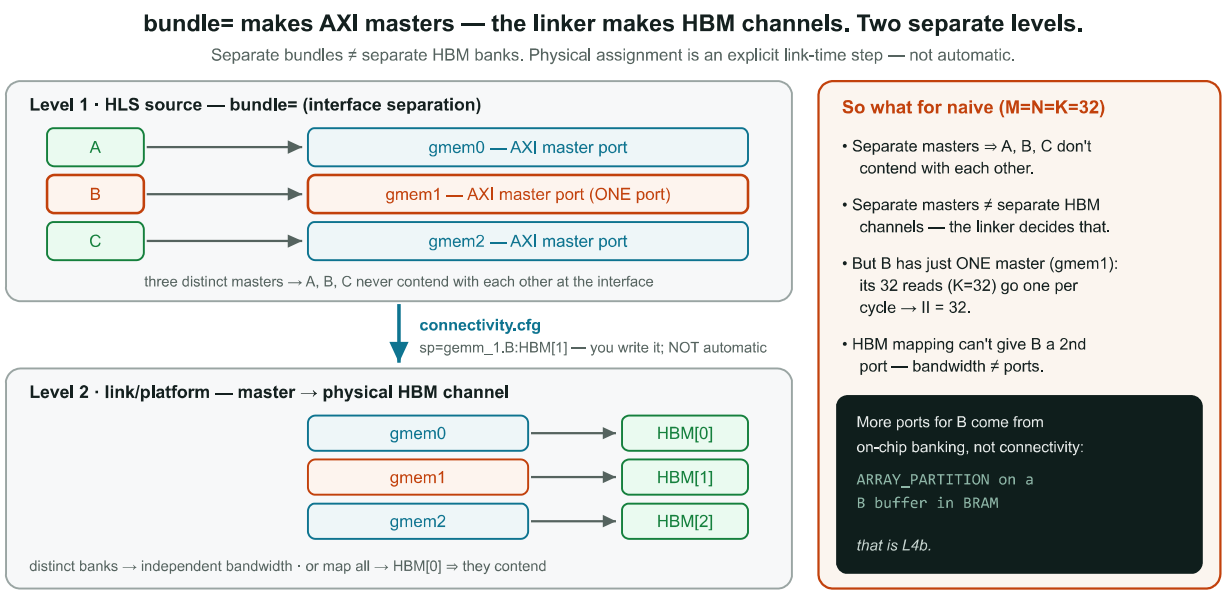
Before any pragma, know how the hardware *delivers* data, because that — not the multiplier count — caps the parallelism you can feed. A memory access is a **transaction** that returns a **contiguous burst**; the port is **wide** (a 512-bit word is 16 `float` s), so one transaction is a **vector**, and that vector is what feeds parallel MAC lanes. **You cannot compute in parallel faster than you can fetch in parallel** — so memory layout is the *precondition* for parallelism, not a separate concern.



Layout feeds parallelism: contiguous + wide (ARRAY_RESHAPE) and banked (ARRAY_PARTITION) manufacture many operands/cycle — L4b.

The naive kernel — arrays are memories

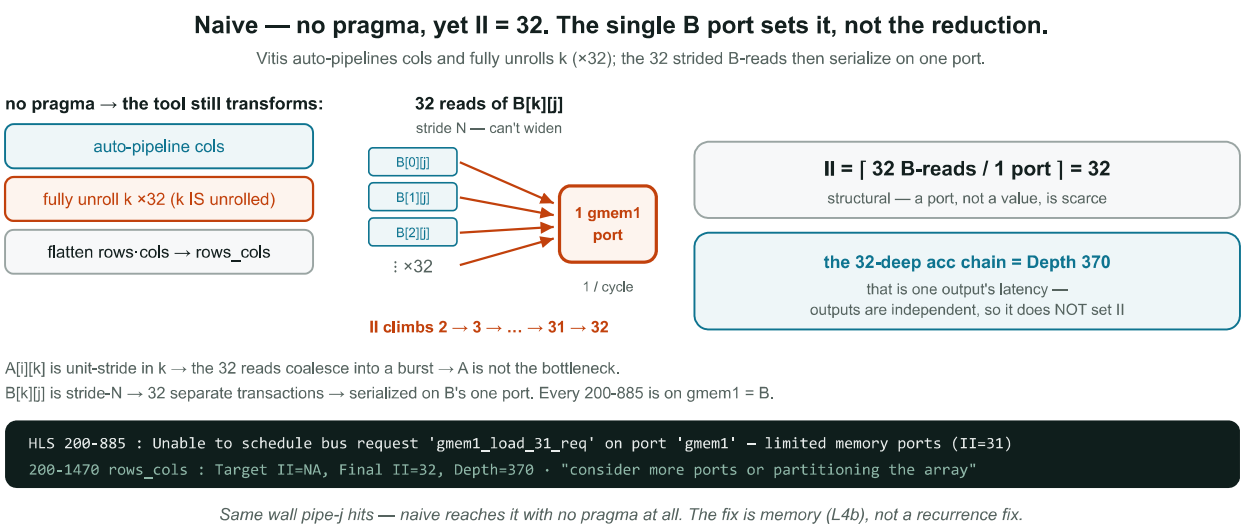
In the naive kernel each array is reached only through an `m_axi` port: `A→gmem0` , `B→gmem1` , `C→gmem2` . Two facts matter. First, **arrays are memories, not variables** — every `A[…]` / `B[…]` is a *port access*, not a register read, so ports (not arithmetic) will bound us. Second, a `bundle=` makes a separate AXI **master**, but which physical **HBM channel** each master uses is a *separate, explicit link-time choice* — and it still cannot give B a second port.



Why A can burst but B cannot. One `m_axi` port serves **one address per cycle**. `A[i*K+k]` is **unit-stride** in k, so its 32 reads coalesce into a burst (`214-115` on `gmem0`) — A is not the bottleneck. `B[k*N+j]` is **stride-N** in k, so each read is a **separate transaction** and B's single master serves **one element per cycle**. That is the scarce resource.

S0 — the naive floor: II = 32, and it is *not* the reduction

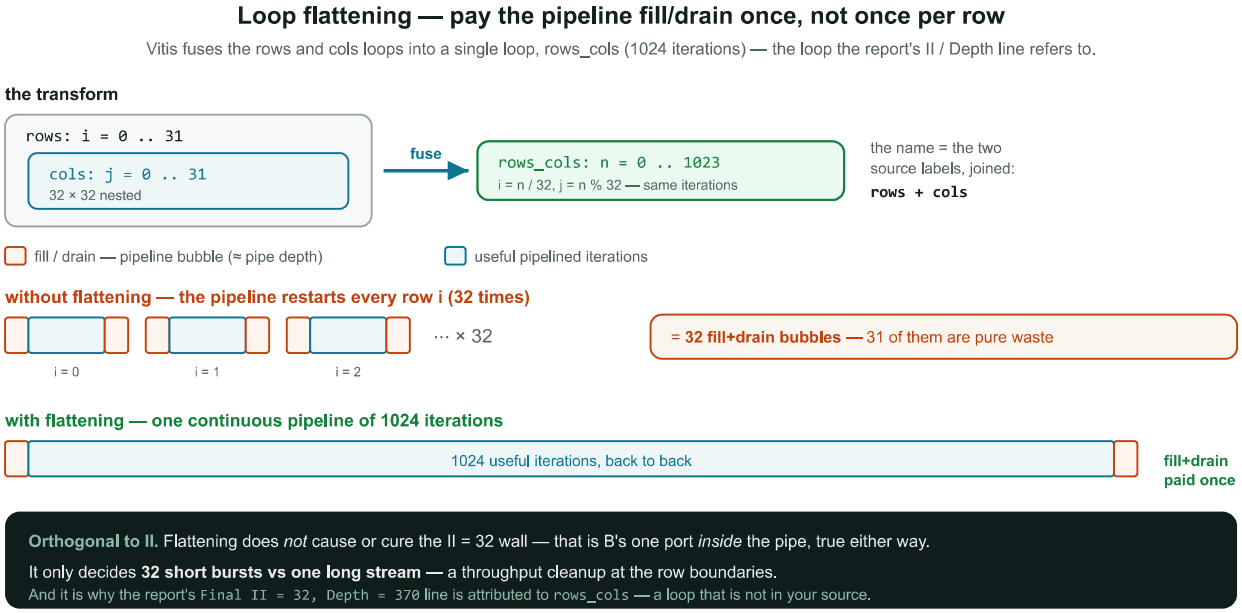
Even with **no pragma**, Vitis auto-pipelines the `cols` loop, **fully unrolls** `k` , and reads the report. The achieved **II = 32** is a **structural / memory-port** hazard: the 32 unrolled, strided reads of B contend for its one `gmem1` port. The log climbs one II per stranded read (`[200-885] … 'gmem1_load_31_req' … limited memory ports (II = 31)` → **Final II = 32**), and prints the fix itself: *"consider a memory core with more ports, or partition the array."* The 32-deep serial float-add chain is the **`Depth = 370`** — one output's *latency* — and because outputs are independent it never sets II.



Same wall pipe-j hits — naive reaches it with no pragma at all. The fix is memory (L4b), not a recurrence fix.

Loop coalescing — what `rows_cols` is

The report's II line names a loop called `rows_cols` that you never wrote. That is **loop coalescing**: Vitis fuses the `rows` and `cols` loops into one loop of $32 \times 32 = 1024$ iterations, reconstructing `i = n/32` and `j = n%32` inside (in hardware, two wrap-around counters — no divide). It fuses them so the pipeline pays its fill/drain **once**, not once per row. Coalescing is **orthogonal to the II = 32 wall** (that is B's port, either way) — it only decides whether that pipeline runs as 32 short bursts or one long stream.

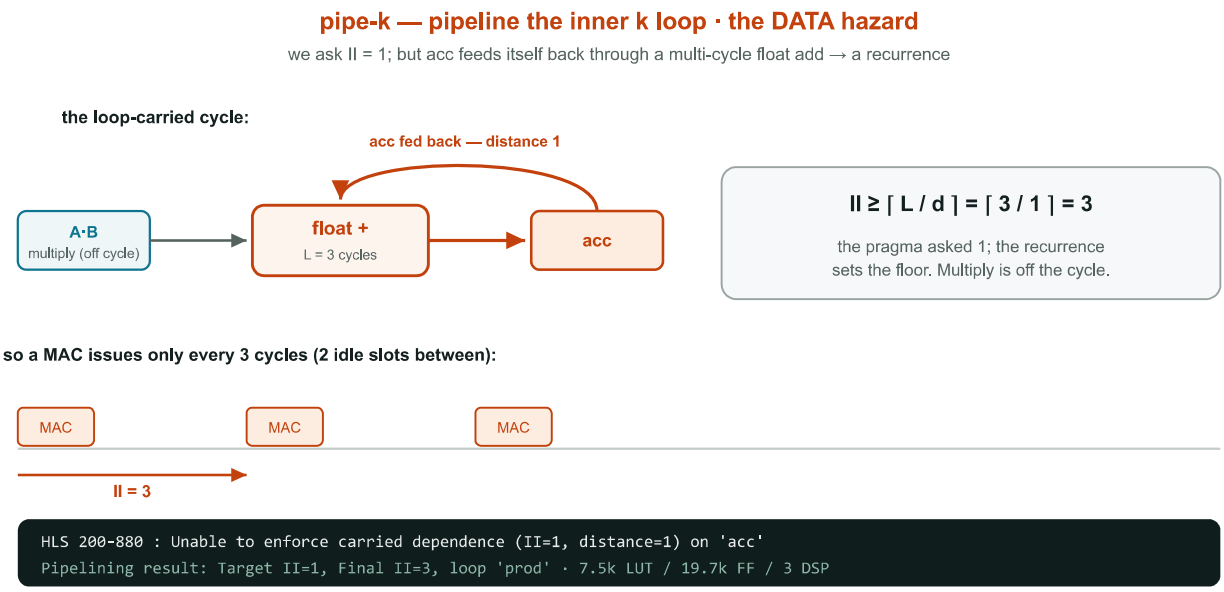


The floor is set: **II = 32, structural, B's one port.** The report's parting advice — *more ports, or partition the array* — is the brief for the whole rest of the lecture. But first, the two pragma instincts everyone tries.

Part V — The design-space walk: two pragma instincts, two hazards

S1 — pipe-k: pipeline the reduction → the DATA hazard

Instinct 1: put `PIPELINE II=1` on the inner `k` loop. But `acc` feeds itself back through a **3-cycle float add** — a recurrence of distance 1 — so $\text{II} \ge \lceil L/d \rceil = \lceil 3/1 \rceil = \mathbf{3}$. This is **L3, unchanged, inside GEMM**: a *value* isn't ready in time. Its two L3 fixes both reach II = 1 (interleaved **partial sums** raise the distance; **fixed-point** lowers the latency) — but they fix *only this*, and we still pound B's single port.

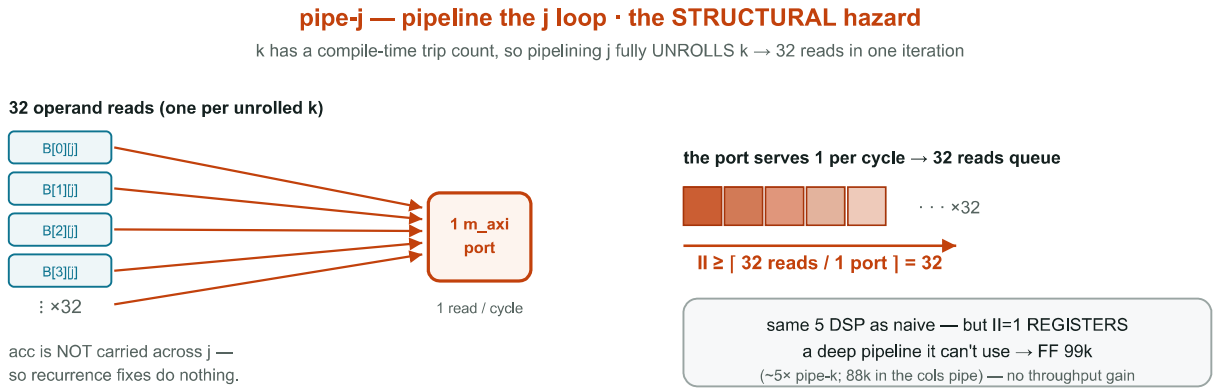


S2 — pipe-j: one output per iteration → the STRUCTURAL hazard

Instinct 2: pipeline the `j` loop instead, so "one output finishes per iteration." Because `k` has a compile-time trip count, pipelining `j` **fully unrolls `k`**, which issues **32 reads of B through one port** → $\text{II} \ge \lceil 32/1 \rceil = \mathbf{32}$ — the *same*

`gmem1` wall naive already hit. A **port**, not a value, is scarce, so recurrence fixes do nothing.

The instructive part is the cost. pipe-j uses the **same 5 DSPs as naive** (one MAC, time-shared) — so the extra cost is **registers, not arithmetic**. Asking `II = 1` makes the tool build a deeply pipelined datapath *shaped* for one column per cycle; 88k of its 99k flip-flops sit in that `cols` pipeline. The single B-port then throttles the *achieved* II back to 32, leaving those registers stranded: **~5× the FF of naive, for the same II.**



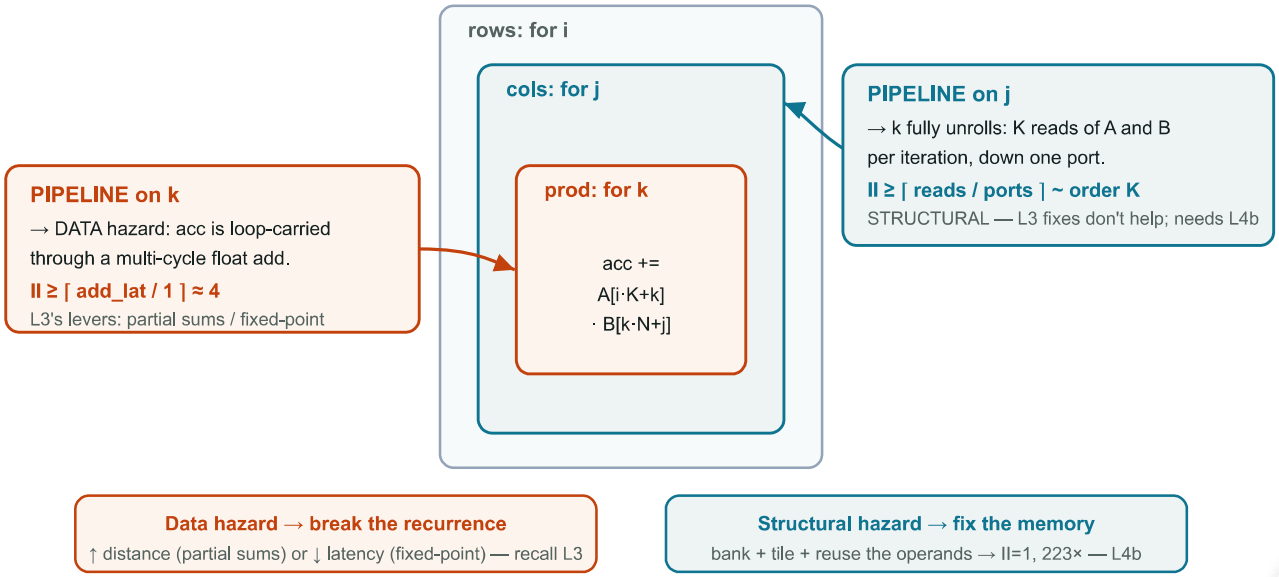

```
Pipelining result: Target II=1, Final II=32, loop 'cols' (II = K)
15.8k LUT / 99.0k FF / 16 BRAM / 5 DSP · vs pipe-k 7.5k / 19.7k
```

Achieved II is a max of three bounds

$$\text{II}_{\text{achieved}} = \max\big(\, \text{requested},\ \underbrace{\lceil L/d \rceil}_{\text{recurrence}},\ \underbrace{\lceil \text{uses}/\text{ports} \rceil}_{\text{structural / resource}} \big)$$

"Just add PIPELINE" moves only the *requested* term. The data hazard you can fix with L3's tools; the **structural / port** hazard you cannot — it needs a change of memory, and that is the rest of the lecture.

Same naive GEMM — where you put PIPELINE decides which hazard you hit



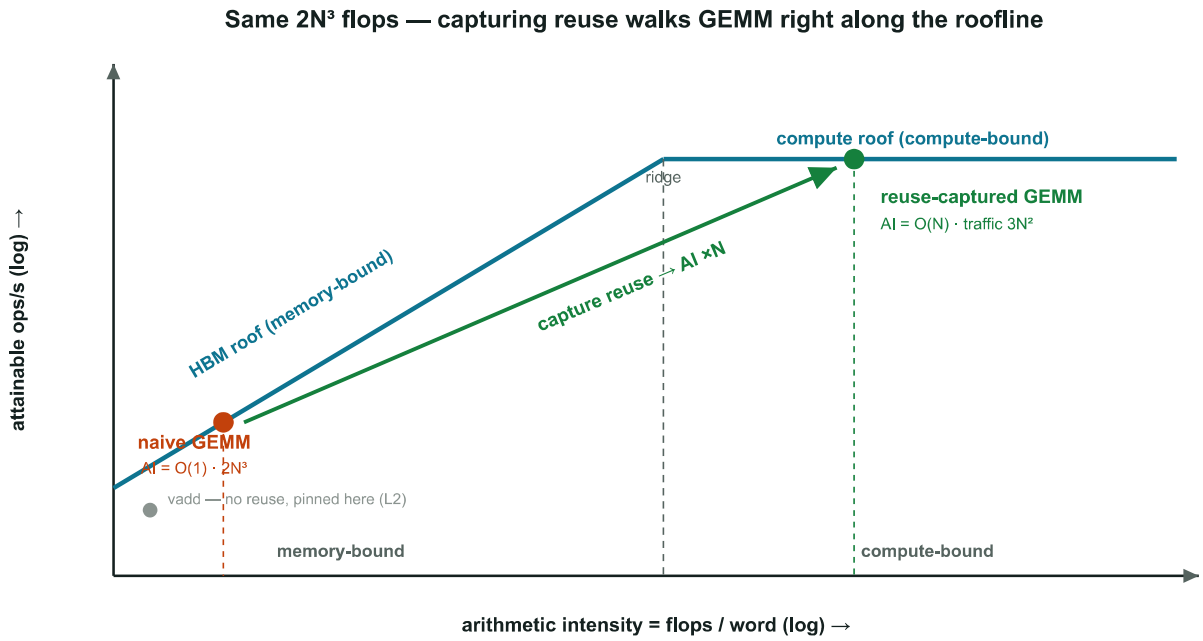
The scoreboard (U50, Vitis 2025.1, 32^3, all cosim-PASS)

Kernel	The one pragma	Achieved II	LUT / FF	Bound that bit
`naive`	none	**32**	11.1k / 12.9k	structural — 32 B-reads / 1 port (k auto-unrolls)
`pipe-k`	`PIPELINE k`	**3** (asked 1)	7.5k / 19.7k	**data** — float `acc`, distance 1
`pipe-j`	`PIPELINE j` (k unrolls)	**32**	15.8k / **99.0k**	structural — K reads / port
`partial-sums`	`PIPELINE k`, 8 lanes	**1** ✓	7.9k / 8.0k	recurrence broken (↑ distance)
`fixed-pipe-k`	`PIPELINE k`, `ap_fixed`	**1** ✓	7.2k / 7.3k	recurrence broken (↓ latency)

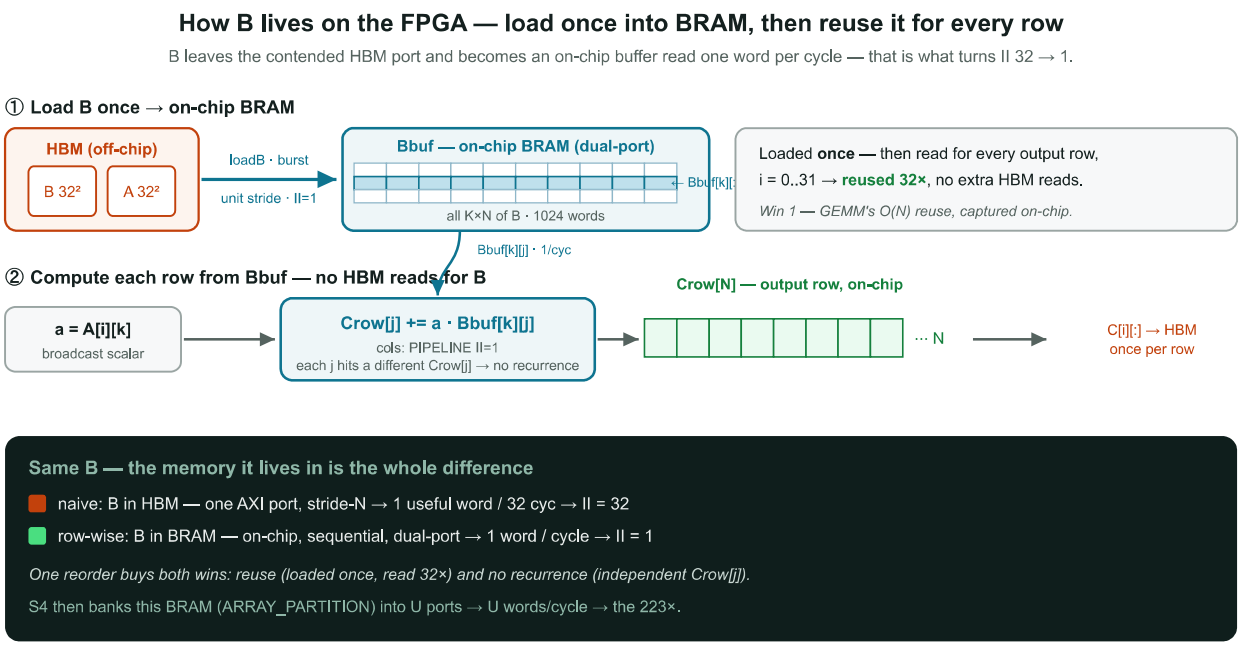
Every row here still **re-reads B from far memory** — none captured GEMM's $O(N)$ reuse, and the structural wall still stands. That is what the next strategy changes.

Part VI — The dataflow that wins: row-wise → II = 1 (measured)

Both walls trace to one root: **the naive dataflow throws away GEMM's reuse and hammers one far-memory port.** Capture the reuse and the traffic drops from $\sim 2N^3$ (naive) to $\sim 3N^2$ — a factor of N — moving the kernel off the memory roof.



S3 — row-wise, buffered. Reorder to i,k,j, load all of B **once** into an on-chip array (which HLS maps to **BRAM**), hold the output row `Crow` on-chip, broadcast $A[i][k]$, stream B by rows, and pipeline `j`. The whole fix is one declaration — `data_t Bbuf[…]` — because every `Bbuf[…]` is now a fast dual-port on-chip read, not the strided far-memory port that pinned naive at II = 32.



This one move wins **twice**:

- **Reuse captured.** B is loaded once into BRAM and the C-row is held on-chip → traffic $\sim 3N^2$, AI $O(N)$: off the memory roof, and B is no longer a far-memory port.
- **Recurrence dissolved, for free.** The inner loop now sweeps `j`, and each step touches a *different* `Crow[j]` — no tight loop-carried dependence — so it pipelines at **II = 1 even on float**, with no partial sums and no fixed-point. *The reorder that fixed memory fixed the S1 data hazard too.*

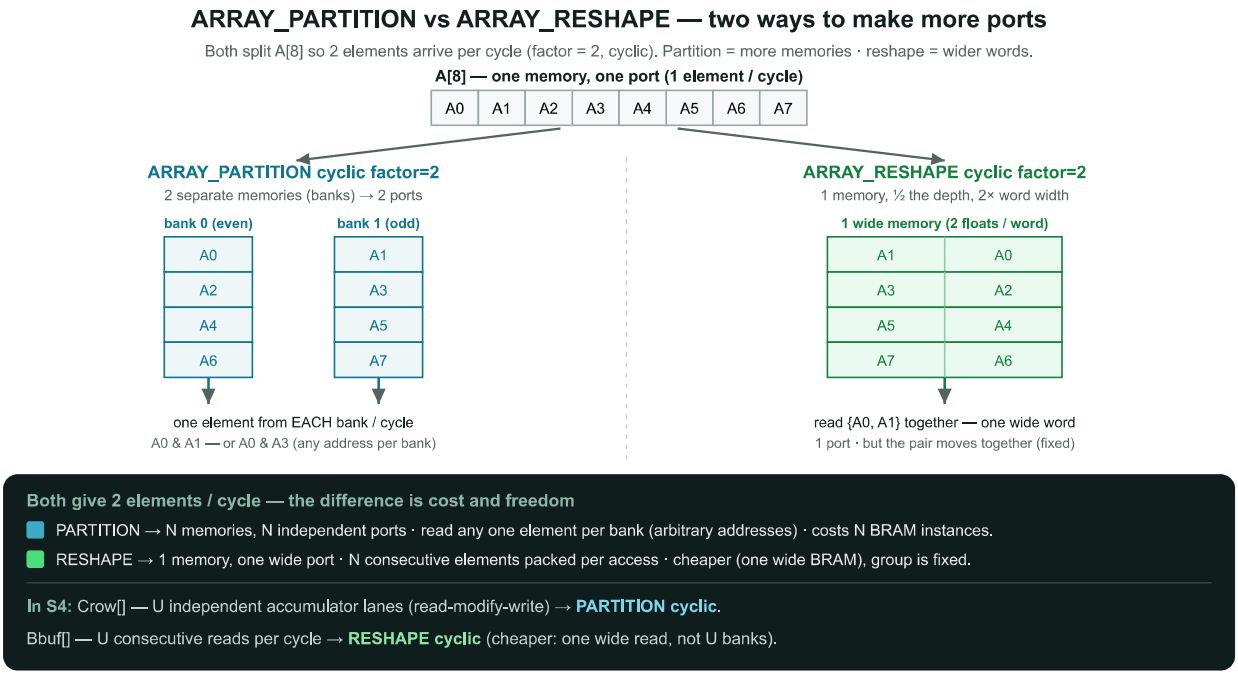
Measured, it is real: the inner loop synthesizes at **II = 1 on float**, total latency **36,817 cycles** for 32^3 versus naive's cosim-measured **143,652** — about **4×** — at a *fraction* of pipe-j's FF (**9.7k LUT / 11.3k FF**), plus **18 BRAM**. That BRAM *is* the reuse — the price that buys II = 1. And unlike naive, whose optimistic csynth estimate (33k) was exposed by cosim (143k), row-wise's estimate matches its cosim, because with B on-chip the strided stall is gone.

Part VII — Memory customization: the width lever

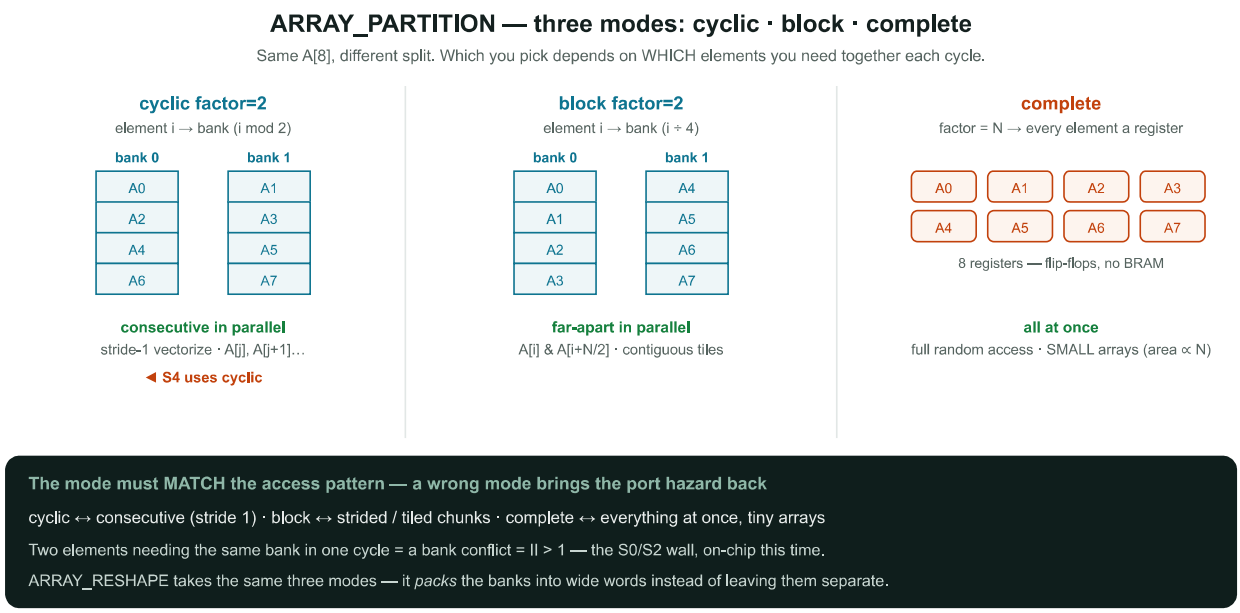
Row-wise reaches II = 1 but still retires **one** result per cycle. Now that B is on-chip, we can **add ports** — the one thing no far-memory pragma could — and retire U

results per cycle. Two pragmas do two different jobs, and choosing correctly is the whole skill.

- `ARRAY_PARTITION` splits one array into **N separate memories (banks)** → N independent ports, any address per bank, at the cost of N BRAM instances.
- `ARRAY_RESHAPE` folds the array into **one wider memory** → one port that delivers **N consecutive elements packed** per access — cheaper, but the group moves together.



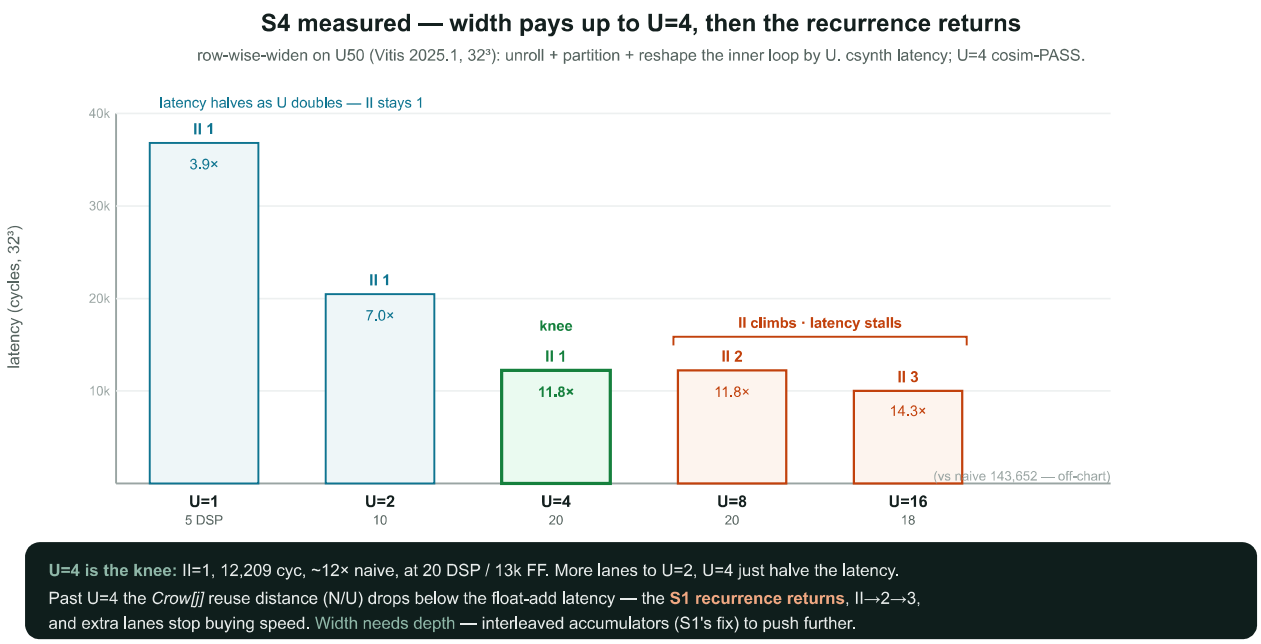
Both take a **cyclic / block / complete** mode, and the mode must match the access pattern or the port hazard comes right back on-chip: **cyclic** interleaves (consecutive elements go to different banks — right for stride-1 vectorization), **block** groups contiguous chunks, **complete** gives every element its own register (small arrays only). In our widened kernel `Crow` is `ARRAY_PARTITION cyclic` (U independent accumulator lanes) and `Bbuf` is `ARRAY_RESHAPE cyclic` (U consecutive reads per cycle).



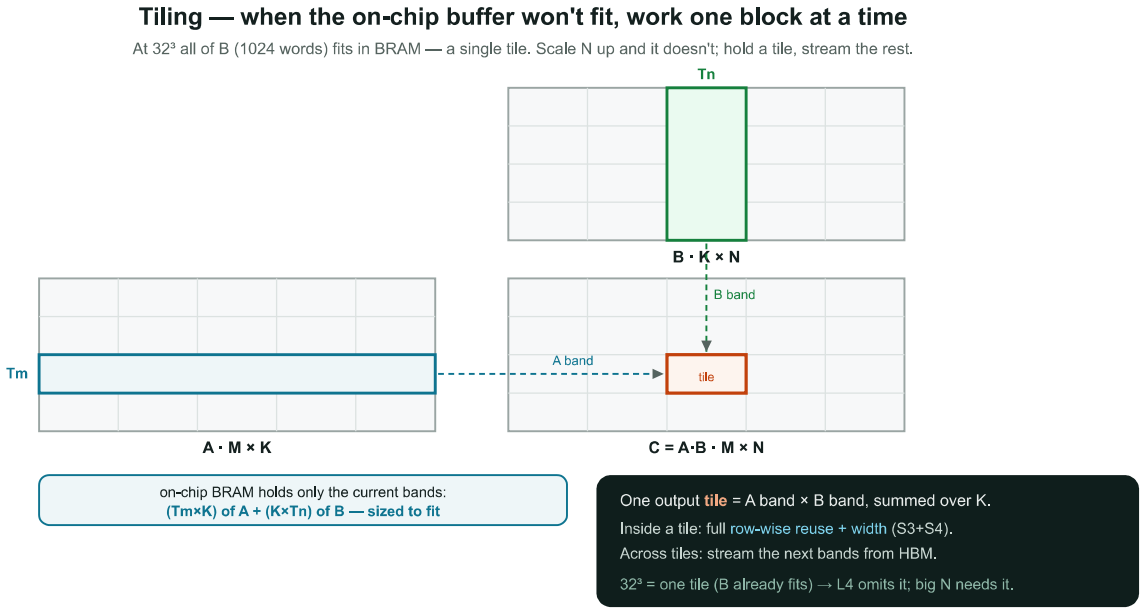
The measured width sweep. Unrolling `cols` by U (with the partition/reshape above) gives a real curve on the U50:

U	inner II	latency (cyc)	DSP	vs naive
1	1	36,817	5	3.9×
2	1	20,433	10	7.0×
4	**1**	**12,209**	20	**11.8×**
8	2	12,209	20	11.8×
16	3	10,033	18	14.3×

Width scales cleanly to **$U = 4$ (II = 1, ~12× naive)** — then a **new wall**. Past the knee, each `Crow[j]` is revisited every N/U cycles; at $U = 8$ that distance (4) drops below the float-add latency, so the **S1 recurrence returns**, II climbs to 2 then 3, and the extra lanes stop buying speed. To push further you need **depth × width** — interleaved partial accumulators (S1's own fix) on top of the width.



Tiling. At 32^3 all of B (1024 words) fits in BRAM — a single tile. Scale N up and it will not; then you hold a $T_m \times T_n$ **tile** (bands of A and B) sized to fit BRAM, reuse fully inside the tile, and stream the next bands from HBM.



The through-line. Reuse (S3) *enabled* on-chip banking; width (S4) *spends* it. Off the memory roof, width climbs the compute roof until the next bound bites — first the `Crow` recurrence (fixable), then the finite supply of DSPs. Our best measured result at 32^3 is **~12×**, on our own U50 numbers; the deeper customization that reaches the

headline speedups at scale — expressing schedules as a *language* — is the subject of the next lecture.

Where this goes next

- **L1** built the datapath · **L2** pipelined it and drew the roofline · **L3** made the pipeline survive a real dependency.
- **L4** scaled the dot product to GEMM, walked **four strategies**, hit **two walls**, and tore both down — **II = 1 on float**, then **~12× at width** — until the recurrence returned and pointed back at L3's fix.
- The **next lecture** treats these same schedules as a *language* (reorder / buffer / partition expressed declaratively); the **outer-product / stationary** view leads to the systolic array later still.

Same $2N^3$ flops throughout; the II and the memory bill were a **dataflow choice**.

References

- **Allo** (PLDI 2024) — the GEMM story from the inner-product formulation through memory customization; the anchor for the next lecture.
- **L2 roofline** (Lecture 2 handout, Part IV) — arithmetic intensity and the reuse argument this lecture quantifies for GEMM.
- **L3 recurrence bound** (Lecture 3 handout) — the data-hazard half of the walk (pipe-k) is L3 applied at GEMM scale.
- **Bluebook, Chapter 4** — loop pipelining, II, and on-chip buffering; the memory mechanics (`ARRAY_PARTITION` / `ARRAY_RESHAPE`) live here.
- **Goto–van de Geijn, layered GEMM** — the loop-order and blocking analysis of Parts II–III on CPUs; the FPGA version manages the on-chip memory explicitly.